1)

When D is a class: Java classes can only extend one class, so D cannot directly extend both B and C simultaneously. The diamond problem doesn't arise for classes because Java simply forbids multiple class inheritance. D must pick one parent class and implement the other as an interface.

ii. When D is an interface: This is where it gets interesting. If B and C are interfaces with default method implementations /Java 8's behavioral inheritance, and D is also an interface that extends both, Java requires D to **explicitly override** method() to resolve the ambiguity. If D doesn't override it, the compiler throws an error. If D is a class instead, same rule applies it must override method() itself, or explicitly call one parent's version using B.super.method() or C.super.method()

2)

```
public sealed interface Expr permits Constant, Add, Multiply {
    int eval();
}
public record Constant(int value) implements Expr {
    public int eval() {
        return value;
    }
}
public record Add(Expr left, Expr right) implements Expr {
    public int eval() {
        return left.eval() + right.eval();
    }
}
public record Multiply(Expr left, Expr right) implements Expr {
    public int eval() {
        return left.eval() * right.eval();
    }
```

```
}
public class Main {
    public static void main(String[] args) {
        // (2 + 3) * 4
        Expr expr = new Multiply(
            new Add(new Constant(2), new Constant(3)),
            new Constant(4)
        );
        System.out.println(eval(expr)); // prints 20
    }

    public static int eval(Expr expr) {
        return switch (expr) {
            case Constant c  -> c.value();
            case Add a       -> eval(a.left()) + eval(a.right());
            case Multiply m  -> eval(m.left()) * eval(m.right());
        };
    }
}
```